torch.linalg.lu_solve#

https://pytorch.org/docs/stable/generated/torch.linalg.lu_solve.html

Description:

Computes the solution of a square system of linear equations with a unique solution given an LU decomposition. Letting K\mathbb{K}K be R\mathbb{R}R or C\mathbb{C}C, this function computes the solution X∈Kn×kX \in \mathbb{K}^{n \times k}X∈Kn×k of the linear system associated to A∈Kn×n,B∈Kn×kA \in \mathbb{K}^{n \times n}, B \in \mathbb{K}^{n \times k}A∈Kn×n,B∈Kn×k, which is defined as where AAA is given factorized as returned by lu_factor(). If left= False, this function returns the matrix X∈Kn×kX \in \mathbb{K}^{n \times k}X∈Kn×k that solves the system If adjoint= True (and left= True), given an LU factorization of AAA this function function returns the X∈Kn×kX \in \mathbb{K}^{n \times k}X∈Kn×k that solves the system

Function Signature

```
torch.linalg.lu_solve(LU, pivots, B, *, left=True,
adjoint=False, out=None) → Tensor#

Parameters

Keyword Arguments
```

Parameters

Keyword

left (bool, optional) – whether to solve the system AX=BAX=BAX=B or XA=BXA = BXA=B. Default: True. adjoint (bool, optional) – whether to solve the system AX=BAX=BAX=B or AHX=BA^{\text{H}}X = BAHX=B. Default: False. out (Tensor, optional) – output tensor. Ignored if None. Default: None.

Code Examples

```
>>> A = torch.randn(3, 3)
>>> LU, pivots = torch.linalg.lu_factor(A)
>>> B = torch.randn(3, 2)
>>> X = torch.linalg.lu_solve(LU, pivots, B)
>>> torch.allclose(A @ X, B)
True

>>> B = torch.randn(3, 3, 2)   # Broadcasting rules apply: A is
broadcasted
>>> X = torch.linalg.lu_solve(LU, pivots, B)
>>> torch.allclose(A @ X, B)
True

>>> B = torch.randn(3, 5, 3)
>>> X = torch.linalg.lu_solve(LU, pivots, B, left=False)
>>> torch.allclose(X @ A, B)
True

>>> B = torch.randn(3, 3, 4)   # Now solve for A^T
>>> X = torch.linalg.lu_solve(LU, pivots, B, adjoint=True)
>>> torch.allclose(A.mT @ X, B)
True
```

Detailed Documentation

Documentation

Computes the solution of a square system of linear equations with a unique solution given an LU decomposition.

Letting K\mathbb{K}K be R\mathbb{R}R or C\mathbb{C}C, this function computes the solution X∈Kn×kX \in \mathbb{K}^{n \times k}X∈Kn×k of the linear system associated to A∈Kn×n,B∈Kn×kA \in \mathbb{K}^{n \times n}, B \in \mathbb{K}^{n \times k}A∈Kn×n,B∈Kn×k, which is defined as

where AAA is given factorized as returned by lu_factor().

If left= False, this function returns the matrix X∈Kn×kX \in \mathbb{K}^{n \times k}X∈Kn×k that solves the system

If adjoint= True (and left= True), given an LU factorization of AAA this function function returns the X∈Kn×kX \in \mathbb{K}^{n \times k}X∈Kn×k that solves the system

where AHA^{\text{H}}AH is the conjugate transpose when AAA is complex, and the transpose when AAA is real-valued. The left= False case is analogous.

Supports inputs of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if the inputs are batches of matrices then the output has the same batch dimensions.

LU (Tensor) – tensor of shape (*, n, n) (or (*, k, k) if left= True) where * is zero or more batch dimensions as returned by lu_factor().

pivots (Tensor) – tensor of shape (*, n) (or (*, k) if left= True) where * is zero or more batch dimensions as returned by lu_factor().

B (Tensor) – right-hand side tensor of shape (*, n, k).

left (bool, optional) – whether to solve the system AX=BAX=BAX=B or XA=BXA = BXA=B. Default: True.

adjoint (bool, optional) – whether to solve the system AX=BAX=BAX=B or AHX=BA^{\text{H}}X = BAHX=B. Default: False.

out (Tensor, optional) – output tensor. Ignored if None. Default: None.